# **APLIKASI KOMPUTER** dan **BASIS DATA**

PERTEMUAN 2: DATABASE INTRODUCTION

PRAKTISI MENGAJAR 2023
ANDRI NUR ARIEF (TC 08) - MUHAMMAD ADITYA RIDHARRAHMAN (TC 10)

# Pertemuan 2 – Database Introduction

- What is Database
- Database variations
- What is RDBMS
- PostgreSQL and DBeaver introduction
- PostgreSQL and DBeaver Installation check
- DDL and DML
- Exercise database creation
- QnA

# What is Database

A database is a structured and organized collection of data that is stored electronically in a computer or a similar system.

It is designed to efficiently store, manage, and retrieve data for various purposes, including data retrieval, modification, and analysis.

Databases are integral to many aspects of modern computing and play a crucial role in numerous applications, such as web applications, business systems, scientific research, and more.

# Database Variations - (1)

- **Relational Databases**
  These databases use a structured format with tables and predefined relationships between data elements. Examples include MySQL, PostgreSQL, Oracle, Microsoft SQL Server.

- **NoSQL Databases**
  NoSQL databases are designed to handle unstructured or semi-structured data and are categorized into several subtypes:
    - **Document Stores:** Examples include MongoDB, CouchDB.
    - **Key-Value Stores:** Examples include Redis, Riak.
    - **Column-Family Stores:** Examples include Apache Cassandra, HBase.

- **NewSQL Databases**
  A category that combines the benefits of traditional relational databases with the scalability of NoSQL. Examples include Google Spanner, CockroachDB.

# Database Variations - (2)

- **In-Memory Databases**
  These databases store data in RAM for extremely fast data access. Examples include Redis, Memcached.

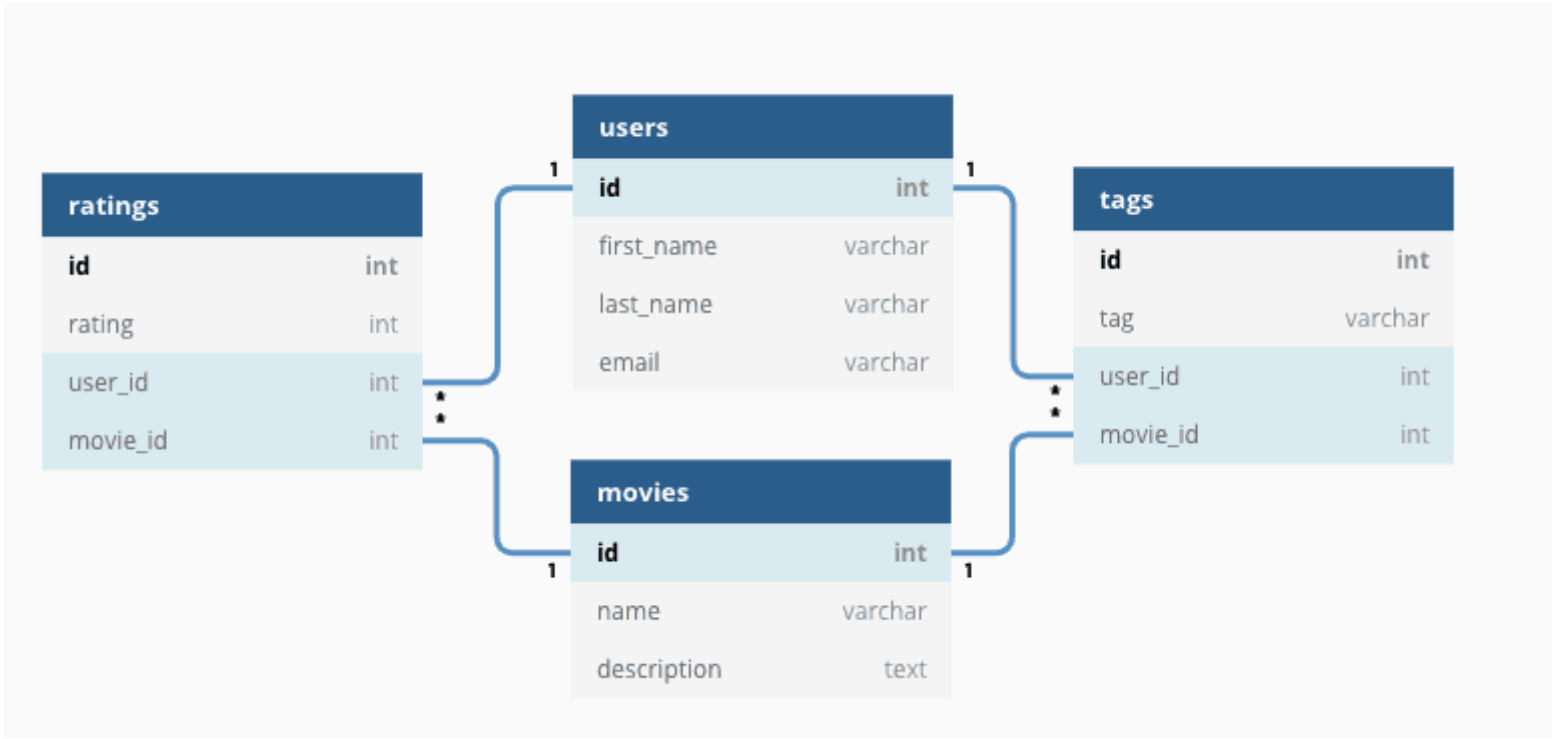
- **Time Series Databases**
  Designed for storing and analyzing time-stamped data, often used in IoT and monitoring applications. Examples include InfluxDB, TimescaleDB.

- **Spatial Databases**
  Optimized for storing and querying spatial or geographic data. Examples include PostGIS (an extension for PostgreSQL), Oracle Spatial.

- **Distributed Databases**
  These databases are distributed across multiple servers or nodes to handle large-scale data processing. Examples include Amazon DynamoDB, Google Bigtable.

- **Mobile Databases**
  Databases optimized for mobile applications and devices. Examples include SQLite, Realm

# Relational Database



# No SQL Database










```
{
  "_id": "5cf0029caff5056591b0ce7d",
  "firstname": "Jane",
  "lastname": "Wu",
  "address": {
    "street": "1 Circle Rd",
    "city": "Los Angeles",
    "state": "CA",
    "zip": "90404"
  }
  "hobbies": ["surfing", "coding"]
}
```

Phone directory

| Key | Value |
|---|---|
| Paul | (091) 9786453778 |
| Greg | (091) 9686154559 |
| Marco | (091) 9868564334 |

MAC table

| Key | Value |
|---|---|
| 10.94.214.172 | 3c:22:fb:86:c1:b1 |
| 10.94.214.173 | 00:0a:95:9d:68:16 |
| 10.94.214.174 | 3c:1b:fb:45:c4:b1 |

```
Please set a database with the command "use <database>".
> use telegraf
Using database telegraf
> SELECT * FROM "cpu_percentage" WHERE time > now() - 5m
> SELECT * FROM "cpu_percentage" WHERE time > now() - 30m
name: cpu_percentage
time                host      process     system
----                ----      -------     ------
1520382106002000000 AmazonBay 0.00159508  0.100567
1520382109003000000 AmazonBay 0.0048583   0.0874694
1520382111001000000 AmazonBay 0.00534394  0.100121
1520382115001000000 AmazonBay 0.000697975 0.116881
1520382118000000000 AmazonBay 0.000604293 0.101674
1520382120000000000 AmazonBay 0.000497881 0.107501
1520382123004000000 AmazonBay 0.000567829 0.0923773
1520382127000000000 AmazonBay 0.000390385 0.162061
1520382129003000000 AmazonBay 0.000363968 0.0973577
1520382133000000000 AmazonBay 0.000679683 0.232044
1520382138004000000 AmazonBay 0.000461529 0.20971
1520382142000000000 AmazonBay 0.000467662 0.275942
1520382144002000000 AmazonBay 0.000477053 0.167299
1520382148000000000 AmazonBay 0.000523114 0.105673
1520382150000000000 AmazonBay 0.000447128 0.107531
1520382154004000000 AmazonBay 0.000510248 0.0886628
1520382157000000000 AmazonBay 0.000461832 0.0959543
1520382159000000000 AmazonBay 0.000155011 0.0975067
1520382162000000000 AmazonBay 0.00339232  0.0699947
1520382166005000000 AmazonBay 0.000459554 0.0905126
1520382168005000000 AmazonBay 0.000739153 0.0700027
1520382171001000000 AmazonBay 0.000284474 0.0826134
1520382174003000000 AmazonBay 0.000645332 0.102407
1520382178002000000 AmazonBay 0.000452395 0.107534
1520382181004000000 AmazonBay 0.000531506 0.107417
1520382184004000000 AmazonBay 0.000784396 0.0891832
1520382186001000000 AmazonBay 0.000439439 0.211582
1520382190000000000 AmazonBay 0.000476813 0.129392
1520382192005000000 AmazonBay 0.000996028 0.110989
1520382195003000000 AmazonBay 0.000510438 0.0858931
1520382199004000000 AmazonBay 0.000485735 0.055609
1520382201001000000 AmazonBay 0.000164502 0.0663518
1520382205001000000 AmazonBay 0.000539746 0.147482
1520382208004000000 AmazonBay 0.000610359 0.0974243
```

# Relational Database Use Case

Relational databases are well-suited for a wide range of use cases, and they are particularly useful in scenarios that require strong data consistency, structured data, and complex relationships between data elements. Here are situations where you might consider using a relational database

**Joins and Relationships:** If your data involves complex relationships and associations, such as many-to-many or one-to-many relationships, relational databases are well-suited for modeling these connections. This is common in applications with customer data, product catalogs, or organizational structures.

**Data Integrity and Consistency:** When maintaining data integrity and ensuring the consistency of data is crucial, relational databases, which adhere to the ACID (Atomicity, Consistency, Isolation, Durability) properties, are a strong choice. This is important in applications such as financial systems, healthcare, and e-commerce where data accuracy is paramount.

**Structured Data:** Relational databases are designed for structured data that can be organized into tables with well-defined relationships. If your data can be clearly categorized into entities with attributes, relational databases are a good fit

# No SQL Use Case

### MongoDB

**Flexible Schema:** MongoDB uses a flexible, schema-less data model, which means you can store data without a predefined structure. This flexibility is useful when dealing with evolving or rapidly changing data where the schema is not well-defined upfront.

**Big Data and High Volume:** MongoDB can efficiently handle large volumes of data and high-velocity data streams, making it suitable for applications involving real-time data ingestion and analysis, such as IoT (Internet of Things) or sensor data.

### Redis (Remote Dictionary Server)

**Caching:** Redis is excellent for caching frequently accessed data, such as web page content, database query results, or API responses. It can significantly reduce the response time for read-heavy applications.

**Session Store:** Redis is commonly used as a session store to manage user sessions in web applications. Its in-memory storage and data structure capabilities make it well-suited for this purpose.

**Leaderboards and Counting:** Redis's sorted sets and increment operations make it suitable for implementing leaderboards, tracking user engagement metrics, and counting operations.

**Rate Limiting:** Redis can help implement rate limiting for APIs and services to prevent abuse and ensure fair usage.

# What is RDBMS

- The software used to store, manage, query, and retrieve data stored in a relational database is called a relational database management system (RDBMS).

- A relational database is a type of database that stores and provides access to data points that are related to one another.

- RDBMS is the basis for all modern database systems such as MySQL, Microsoft SQL Server, Oracle, and PostgreSQL

- RDBMS uses SQL queries to access the data in the database.

# What is PostgreSQL

PostgreSQL, often referred to as "Postgres," is a powerful, open-source relational database management system (RDBMS). It is known for its robustness, reliability, and advanced features, making it one of the most popular RDBMS choices for a wide range of applications, from small projects to large-scale enterprise systems.

PostgreSQL is widely used in a variety of applications, including web applications, data warehousing, GIS (Geographic Information Systems), financial systems, and more. Its reputation for stability and its commitment to open-source development have made it a popular choice for organizations and developers seeking a powerful and reliable RDBMS.

# PostgreSQL vs MySQL

PostgreSQL and MySQL are both popular open-source relational database management systems (RDBMS) that offer similar features but have some differences in terms of functionality, performance, and use cases. Here's a comparison of PostgreSQL and MySQL:

**Licensing:**

- PostgreSQL: PostgreSQL uses the PostgreSQL License, which is a permissive open-source license.
- MySQL: MySQL uses the GNU General Public License (GPL) or commercial licenses, depending on the edition.

•**Ease of Use:**

- MySQL is often considered more user-friendly and easier to set up for beginners.
- PostgreSQL may be seen as more developer-focused, providing powerful features and customization options but potentially having a steeper learning curve.

# What is DBeaver

DBeaver is a popular, open-source, and cross-platform database management tool that provides a universal database client for connecting to and interacting with various database systems. It is designed to simplify database development, administration, and data analysis tasks by offering a single interface to work with multiple database management systems

# RDBMS – ACID properties

ACID is an acronym that stands for
- Atomicity
- Consistency
- Isolation
- Durability.

These properties are crucial in the context of database management systems (DBMS) and are designed to ensure the reliability and integrity of transactions when working with relational databases.

# Atomicity

- **Definition:** Atomicity ensures that a transaction is treated as a single, indivisible unit. It means that either all the operations within a transaction are successfully completed, or none of them are executed at all

- **Example:** In a banking application, if you transfer money from one account to another, the withdrawal from one account and the deposit into the other must both succeed or fail as a single, atomic operation.

# Consistency

- **Definition:** Consistency guarantees that a transaction takes the database from one consistent state to another. In other words, a transaction should preserve the integrity of the database and adhere to certain predefined rules and constraints.

- **Example:** In a library database, if you check out a book, the transaction should ensure that the book's availability is updated, and it cannot be checked out by another user until you return it.

# Isolation

- **Definition:** Isolation ensures that the execution of one transaction is isolated from the execution of other transactions. This means that transactions running concurrently should not interfere with each other and that their operations should be invisible to one another until they are committed.

- **Example:** If multiple users are simultaneously making bookings for flights, their transactions should not affect each other. They should not be able to book the same seat or have their transactions interfere in any way until they are successfully completed.

# Durability

- **Definition:** Durability guarantees that once a transaction is successfully completed and committed, its effects on the database persist even in the event of system failures (e.g., power outage, hardware failure). The data changes made by a committed transaction should be permanent.

- **Example:** If you update your email address in an online service and the transaction is committed, your new email address should persist in the database even if the server crashes immediately after the update.

# What is SQL

SQL stands for "Structured Query Language."

It is a domain-specific programming language used for managing and manipulating data stored in relational database management systems (RDBMS).

SQL is the standard language for communicating with and querying databases, and it provides a way to create, retrieve, update, and delete data in a structured, relational format.

# SQL Key Component

Key components of SQL include:

**Data Definition Language (DDL):** DDL commands are used to define and manage the structure of the database. Common DDL commands include **CREATE**, **ALTER**, and **DROP**, which create, modify, or delete tables, indexes, and other database objects.

**Data Query Language (DQL):** SQL includes commands for retrieving data from a database. The primary DQL command is **SELECT**, which allows you to retrieve specific data from one or more database tables.

**Data Manipulation Language (DML):** DML commands are used to manipulate data within the database. The primary DML commands are **INSERT**, **UPDATE**, and **DELETE**, which allow you to add, modify, or remove data from tables.

**Data Control Language (DCL):** DCL commands are used to control access and permissions to the database. Common DCL commands include **GRANT** (to provide permissions) and **REVOKE** (to revoke permissions).

**Data Transaction Language (DTL):** DTL commands are used to control transactions within the database. These include **COMMIT** (to save changes) and **ROLLBACK** (to undo changes made during a transaction).

# Installation Check

# Data Type

| Name | Aliases | Description |
|---|---|---|
| bigint | int8 | signed eight-byte integer |
| bigserial | serial8 | autoincrementing eight-byte integer |
| boolean | bool | logical Boolean (true/false) |
| character [ (n) ] | char [ (n) ] | fixed-length character string |
| character varying [ (n) ] | varchar [ (n) ] | variable-length character string |
| date | | calendar date (year, month, day) |
| double precision | float8 | double precision floating-point number (8 bytes) |
| integer | int, int4 | signed four-byte integer |
| numeric [ (p, s) ] | decimal [ (p, s) ] | exact numeric of selectable precision |
| real | float4 | single precision floating-point number (4 bytes) |
| smallint | int2 | signed two-byte integer |
| smallserial | serial2 | autoincrementing two-byte integer |
| serial | serial4 | autoincrementing four-byte integer |
| text | | variable-length character string |
| time [ (p) ] [ without time zone ] | | time of day (no time zone) |
| time [ (p) ] with time zone | timetz | time of day, including time zone |
| timestamp [ (p) ] [ without time zone ] | | date and time (no time zone) |
| timestamp [ (p) ] with time zone | timestamptz | date and time, including time zone |

source: https://www.postgresql.org/docs/current/datatype.html

# Create Database

The CREATE DATABASE statement allows you to create a new PostgreSQL database.
The following shows the syntax of the CREATE DATABASE statement:

```
CREATE DATABASE database_name
WITH
   [OWNER =  role_name]
   [TEMPLATE = template]
   [ENCODING = encoding]
   [LC_COLLATE = collate]
   [LC_CTYPE = ctype]
   [TABLESPACE = tablespace_name]
   [ALLOW_CONNECTIONS = true | false]
   [CONNECTION LIMIT = max_concurrent_connection]
   [IS_TEMPLATE = true | false ]
```

**OWNER**
Assign a role that will be the owner of the database. If you omit the OWNER option, the owner of the database is the role that you use to execute the CREATE DATABASE statement.

**TEMPLATE**
Specify the template database from which the new database is created. By default, PostgreSQL uses the template1 database as the template database if you don't explicitly specify the template database.

**ENCODING**
Determine the character set encoding in the new database.

**TABLESPACE**
Specify the tablespace name for the new database. The default is the tablespace of the template database.

**CONNECTION LIMIT**
Specify the maximum concurrent connections to the new database. The default is -1 i.e., unlimited. This parameter is useful in the shared hosting environments where you can configure the maximum concurrent connections for a particular database.

**ALLOW_CONNECTIONS**
The allow_connections parameter is a boolean value. If it is false, you cannot connect to the database.

# Create Table

A relational database consists of multiple related tables. A table consists of rows and columns. Tables allow you to store structured data like customers, products, employees, etc.

To create a new table, you use the CREATE TABLE statement. The following illustrates the basic syntax of the CREATE TABLE statement:

```
CREATE TABLE [IF NOT EXISTS] table_name (
   column1 datatype(length) column_contraint,
   column2 datatype(length) column_contraint,
   column3 datatype(length) column_contraint,
   table_constraints
);
```

PostgreSQL includes the following column constraints:

- **NOT NULL** – ensures that values in a column cannot be NULL.
- **UNIQUE** – ensures the values in a column unique across the rows within the same table.
- **PRIMARY KEY** – a primary key column uniquely identify rows in a table. A table can have one and only one primary key. The primary key constraint allows you to define the primary key of a table.
- **CHECK** – a CHECK constraint ensures the data must satisfy a boolean expression.
- **FOREIGN KEY** – ensures values in a column or a group of columns from a table exists in a column or group of columns in another table. Unlike the primary key, a table can have many foreign keys.

# Insert Data

The PostgreSQL INSERT statement allows you to insert a new row into a table.
The following illustrates the most basic syntax of the INSERT statement:

```
INSERT INTO table_name(column1, column2, …)
VALUES (value1, value2, …);
```

To insert multiple rows into a table using a single INSERT statement, you use the following syntax:

```
INSERT INTO table_name (column_list)
VALUES
    (value_list_1),
    (value_list_2),
    ...
    (value_list_n);
```

# Read Data

Let’s start with the basic form of the SELECT statement that retrieves data from a single table.
The following illustrates the syntax of the SELECT statement:

```
SELECT
   select_list
FROM
   table_name;
```

The SELECT statement returns all rows from one or more columns in a table. To select rows that satisfy a specified condition, you use a WHERE clause.

```
SELECT select_list
FROM table_name
WHERE condition
ORDER BY sort_expression
```

# Where Condition

| Operator | Description |
|---|---|
| = | Equal |
| > | Greater than |
| < | Less than |
| >= | Greater than or equal |
| <= | Less than or equal |
| <> or != | Not equal |
| AND | Logical operator AND |
| OR | Logical operator OR |
| IN | Return true if a value matches any value in a list |
| BETWEEN | Return true if a value is between a range of values |
| LIKE | Return true if a value matches a pattern |
| IS NULL | Return true if a value is NULL |
| NOT | Negate the result of other operators |

# Update Data

The PostgreSQL UPDATE statement allows you to modify data in a table. The following illustrates the syntax of the UPDATE statement:

```
UPDATE table_name
SET column1 = value1,
    column2 = value2,
    ...
WHERE condition;
```

# Tugas Pertemuan 2

1. Buatlah table “mahasiswa” dengan field-field sebagai berikut:
    a. Nomor Induk Mahasiswa
    b. Nama
    c. Gender
    d. Alamat
    e. Kota Asal
    f. Tanggal Lahir
    g. Tahun Masuk
    h. Nomor Handphone

2. Tentukan tipe data yang paling sesuai dengan kebutuhan data pada column2 tersebut
3. Isilah tabel tersebut dengan data mahasiswa 1 kelas
4. Row pertama WAJIB di isi oleh data pribadi
5. Buatlah laporan pada Github repository yang sebelumnya sudah dibuat dengan ketentuan

- Buatlah sebuah file baru dengan nama "tugas-pertemuan-2.md"
- Sertakan command perintah untuk membuat table diatas
- Sertakan sample command untuk melakukan insert data
- Sertakan screenshot hasil akhir dari table yang sudah berisikan data

# Delete Data

The PostgreSQL DELETE statement allows you to delete one or more rows from a table.
The following shows basic syntax of the DELETE statement:

```
DELETE FROM table_name
WHERE condition;
```

# Referensi


- https://www.oracle.com/id/database/what-is-a-relational-database/
- https://www.geeksforgeeks.org/acid-properties-in-dbms/
- https://www.geeksforgeeks.org/postgresql-select/